```
                min = array[i];
                rVal = i;
            }
        }
        return rVal;
    }

    private static void sort(int[] array, int startIdx) {
        int posOfMin = idxOfMin(array, startIdx);
        if (posOfMin != startIdx) {
            swap(array, posOfMin, startIdx);
        }
        startIdx++;
        if (startIdx < array.length - 1) {
            sort(array, startIdx);
        }
    }

    public static void sort(int[] array) {
        sort(array, 0);
    }

    public static void main(String[] args) {
        int[] tbl = {2, 1, 8, 6, 5, 4};
        sort(tbl);
        System.out.println(Arrays.toString(tbl));
    }
}
```

Κώδικας 9.13 *Αναδρομική υλοποίηση της selection sort*

Στον κώδικα 9.13 η αναδρομική υλοποίηση της selection sort δίνεται από τη συνάρτηση sort(int[], int). Η δεύτερη παράμετρος προκύπτει για διευκόλυνση της αναδρομικής κλήσης. Παρόμοια με τις δύο προηγούμενες ασκήσεις παρέχεται μια συνάρτηση-περιτύλιγμα, η sort(int[]). Οι υπόλοιπες συναρτήσεις είναι γνωστές καθώς έχουν ήδη συζητηθεί.

Ας δούμε λοιπόν την sort(int[], int). Η παράμετρος startIdx καθορίζει τη θέση του πίνακα από την οποία και μετά, ο πίνακας είναι αταξινόμητος. Αρχικά, η συνάρτηση εντοπίζει τη θέση του μικρότερου ακέραιου στο υπό ταξινόμηση τμήμα του πίνακα, δηλαδή από το startIdx και μετά. Αν το μικρότερο στοιχείο βρίσκεται μετά τη θέση startIdx, τότε ανταλλάσσει το μικρότερο στοιχείο με το στοιχείο στη θέση startIdx. Στη συνέχεια, αυξάνει το startIdx κατά 1 έτσι ώστε να συνεχίσει η ταξινόμηση από την επόμενη θέση στον πίνακα. Αν το startIdx δείχνει πριν την τελευταία θέση, σημαίνει ότι υπολείπεται τμήμα του πίνακα προς ταξινόμηση, οπότε γίνεται η αναδρομική κλήση, διαφορετικά τερματίζεται η αναδρομική κλήση και ο πίνακας είναι ταξινομημένος.

Στη συνέχεια παρουσιάζουμε τον κώδικα ελέγχου.

```
public class SelectionSortTest {

    public SelectionSortTest() {
    }

    /**
     * Test of sort method, of class SelectionSort.
     */
    @Test
    public void testSort() {
        System.out.println("sort");
        Random gen = new Random();
        for (int i = 0; i < 100; i++) {
            int[] array = new int[gen.nextInt(100) + 10];
            for (int j = 0; j < array.length; j++) {
```

```
                array[j] = gen.nextInt();
            }
            SelectionSort.sort(array);
            for (int j = 0; j < array.length - 1; j++) {
                if (array[j + 1] < array[j]) {
                    fail();
                }
            }
        }
    }

}
```

Κώδικας 9.14 *Κώδικας ελέγχου της SelectionSort.sort*

Στον κώδικα 9.14 κατασκευάζουμε 100 πίνακες με μέγεθος μεταξύ 10 και 110. Στη συνέχεια εκχωρούμε σε κάθε θέση ενός πίνακα έναν τυχαίο ακέραιο. Σημειώστε πως η nextInt χωρίς παραμέτρους μπορεί να επιστρέψει οποιαδήποτε ακέραιη τιμή. Αφού γεμίσουμε τον πίνακα με τυχαίες τιμές, προχωρούμε στην ταξινόμησή του. Τέλος, ελέγχουμε αν ο πίνακας είναι σωστά ταξινομημένος. Σε περίπτωση που εντοπίσουμε μικρότερο στοιχείο του πίνακα να έπεται μεγαλύτερου, καλούμε την fail και σημειώνουμε το λάθος.

9.6.5 Δυαδική αναζήτηση

Να αναπτυχθεί αναδρομική συνάρτηση δυαδικής αναζήτησης σε μονοδιάστατο πίνακα ακέραιων. Να κατασκευαστεί κατάλληλος κώδικας ελέγχου με χρήση της Junit.

Λύση
Τον αλγόριθμο δυαδικής αναζήτησης τον έχουμε ήδη συζητήσει και έχουμε δώσει και την μη αναδρομική υλοποίησή του (ενότητα 6.5.2). Έτσι, εδώ θα συζητήσουμε αποκλειστικά την αναδρομική υλοποίησή του.

```
public class BinarySearch {

    private static int binarySearch(int[] tbl, int schElement, int from, int to)
{
        if (from <= to) {
            int mid = from + (to - from) / 2;
            if (tbl[mid] == schElement) {
                return mid;
            }
            if (schElement < tbl[mid]) {
                return binarySearch(tbl, schElement, from, mid - 1);
            }
            return binarySearch(tbl, schElement, mid + 1, to);
        }
        return -1;
    }

    public static int binarySearch(int[] tbl, int schElement) {
        return binarySearch(tbl, schElement, 0, tbl.length - 1);
    }

}
```

Κώδικας 9.15 *Αναδρομική δυαδική αναζήτηση*

Στον κώδικα 9.15 η binarySearch(int[], int, int, int) είναι η αναδρομική συνάρτηση. Οι παράμετροι from και to έχουν προστεθεί για να διευκολύνουν την αναδρομή. Η συνάρτηση-περικάλυμμα καλεί την αναδρομική με παραμέτρους 0 και tbl.length-1 ώστε η αναζήτηση να αφορά καταρχάς όλον τον πίνακα. Μέσα στην αναδρομική συνάρτηση γίνεται έλεγχος εάν το from είναι μικρότερο από το to, δηλαδή αν υπάρχει τμήμα του πίνακα που δεν έχει ελεγχθεί. Αν υπάρχει τέτοιο τμήμα, ενημερώνεται κατάλληλα η mid που δείχνει στο μέσο

του υπό αναζήτηση τμήματος. Στη συνέχεια ελέγχεται αν το μέσον έχει την τιμή αναζήτησης. Αν ναι, η συνάρτηση επιστρέφει το μέσον ως τη θέση του στοιχείου που αναζητούμε. Αν όχι, η συνάρτηση προχωράει σε αναδρομική κλήση. Αν η τιμή του στοιχείου αναζήτησης είναι μικρότερη από την τιμή στο μέσο, η αναδρομική κλήση αφορά το πρώτο μισό του υπό εξέταση τμήματος, διαφορετικά αφορά το δεύτερο μισό.
Στη συνέχεια δίνουμε τον κώδικα ελέγχου.

```
import java.util.Random;
import org.junit.Test;
import static org.junit.Assert.*;

/**
 *
 * @author Lefteris Moussiades
 */
public class BinarySearchTest {

     /**
     * Test of binarySearch method, of class BinarySearch.
     */
    @Test
    public void testBinarySearch() {
        System.out.println("binarySearch");
        Random gen=new Random();
        for (int i=0; i<100; i++) {
            int[] array=new int[gen.nextInt(100)+10];
            for (int j=0; j<array.length; j++) {
                array[j]=gen.nextInt();
            }
            SelectionSort.sort(array);
            int idx=gen.nextInt(array.length);
            idx=idx==array.length?array.length-1:idx;
            int schElement=array[idx];
            int result = BinarySearch.binarySearch(array, schElement);
            assertTrue(array[result]==schElement);
        }
    }
}
```

Κώδικας 9.16 *Έλεγχος της binarySearch*

Στον κώδικα 9.16 κατασκευάζουμε 100 πίνακες ακέραιων με τυχαίο μήκος από 10 έως 110 και τους γεμίζουμε με τυχαίες ακέραιες τιμές. Στη συνέχεια ταξινομούμε κάθε πίνακα και επιλέγουμε την τιμή ενός τυχαίου στοιχείου του. Τέλος καλούμε την binarySearch ώστε να αναζητήσουμε το στοιχείο που γνωρίζουμε ότι όντως υπάρχει στον πίνακα.

9.7 Ασκήσεις προς λύση

1. Να αναπτυχθεί συνάρτηση που υπολογίζει αναδρομικά τον Μέγιστο Κοινό Διαιρέτη σειράς ακέραιων. Να αναπτυχθεί κατάλληλος κώδικας ελέγχου.

2. Να αναπτυχθεί αναδρομική συνάρτηση static void prtSquareRoots(int i, int n) που τυπώνει τη σειρά των τετραγώνων των αριθμών από i έως n.

3. Να αναπτυχθεί αναδρομική συνάρτηση static void prtSquareRoots(int i, int n) που τυπώνει τη σειρά των τετραγώνων των αριθμών από i έως n, στην ίδια γραμμή, χωρισμένα με κόμμα και διάστημα. Μετά την εκτύπωση της σειράς, η συνάρτηση να προετοιμάζει ώστε η επόμενη έξοδος να βγαίνει στην επόμενη γραμμή. Για παράδειγμα, τα τετράγωνα των αριθμών από 0 έως 9 να τυπωθούν ως εξής:

```
0, 1, 4, 9, 16, 25, 36, 49, 64, 81
```

4. Να αναπτύξετε αναδρομική συνάρτηση υπολογισμού του αθροίσματος

$$\sum_{i=1}^{n} \frac{1}{i}$$

5. Πόσες αναδρομικές κλήσεις προκαλεί η κλήση fibonacci(9);

Υπόδειξη: Τοποθετήστε την αναδρομική Fibonacci σε μία κλάση. Ορίστε στην κλάση μια ακέραια στατική μεταβλητή cnt. Αυξήστε την cnt κατάλληλα μέσα στην Fibonacci. Εμφανίστε το αποτέλεσμα.

6. Να αναπτυχθεί αναδρομική συνάρτηση που επιστρέφει τη μέγιστη τιμή ενός πίνακα ακέραιων.

7. Να αναπτυχθεί αναδρομική συνάρτηση που μετράει τους πεζούς χαρακτήρες σε έναν πίνακα χαρακτήρων.

8. Πόσες αναδρομικές κλήσεις απαιτεί η λύση των πύργων του Ανόι για 10 δίσκους;

9. Να αναπτυχθεί αναδρομική συνάρτηση που εμφανίζει όλες τις αναδιατάξεις (permutations) μιας συμβολοσειράς. Για παράδειγμα, έστω η συμβολοσειρά can, οι δυνατές αναδιατάξεις είναι can, cna, acn, anc, nca, nac.

10. Να αναπτυχθεί αναδρομική συνάρτηση που λαμβάνει ως παραμέτρους δύο συμβολοσειρές και εξετάζει αν η δεύτερη περιέχεται στην πρώτη.

11. Ένας τρόπος για τον υπολογισμό της τετραγωνικής ρίζας ενός πραγματικού αριθμού, x>0, έχει ως εξής: Έστω y μια τυχαία λύση. Αν το y υψωμένο στο τετράγωνο προσεγγίζει ικανοποιητικά το x, τότε η τετραγωνική ρίζα του x είναι το y, διαφορετικά αντικαθιστούμε στη θέση του y, το (y+x/y)/2. Υλοποιήστε αναδρομική συνάρτηση υπολογισμού της τετραγωνικής ρίζας πραγματικού αριθμού.

Βιβλιογραφία

[1] “Chapter 2. The Structure of the Java Virtual Machine.” https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-2.html#jvms-2.5 (accessed Oct. 15, 2021).

[2] E. W. Weisstein, “Hofstadter Male-Female Sequences.” https://mathworld.wolfram.com/HofstadterMale-FemaleSequences.html (accessed Sep. 14, 2021).

Κριτήρια αξιολόγησης

Στατικές και τοπικές μεταβλητές

Έστω ο κώδικας

```
public class KA01 {

     static int i = 2;

     public static void main(String[] args) {
          for (int i = 5; i < 10; i++) {
               i++;
          }
          System.out.println(i--);
     }
}
```

Ποια είναι η έξοδός του;

```
A. 56789
B. 5678910
Γ. 2
Δ. 1
Ε. Λάθος χρόνου μεταγλώττισης
```

Απάντηση/Λύση

Οι απαντήσεις Α και Β θα μπορούσαν να απασχολήσουν τον αναγνώστη, εφόσον μέσα στην επαναληπτική διαδικασία τυπώναμε το i. Εδώ το i απλώς αυξάνεται χωρίς να εμφανίζεται. Επιπλέον, η στατική μεταβλητή i είναι διαφορετική από την τοπική i της οποίας η εμβέλεια περιορίζεται στην for. Μεταβολές στην τοπική i δεν επηρεάζουν τη στατική i. Η απάντηση Δ είναι επίσης λανθασμένη. Παρότι η στατική i στέλνεται στην έξοδο συνοδευόμενη από τελεστή προσαύξησης, η προσαύξηση γίνεται αλλά το i-- αξιολογείται στην τιμή πριν την προσαύξηση καθώς χρησιμοποιείται ο μεταθεματικός τελεστής. Ο κώδικας δεν περιέχει λάθος μεταγλώττισης, επομένως και η απάντηση Ε είναι λανθασμένη. Τέλος, η σωστή απάντηση είναι η Γ.

Εμβέλεια τοπικών μεταβλητών

Έστω ο κώδικας

```
public class KA02 {

     public static void main(String[] args) {
          int i = 1;
          do {
               int j = i++;
               System.out.print(j);
          } while (j < 5);
     }
}
```

Ποια είναι η έξοδός του;

```
A. 1234
B. 2345
```

```
Γ. 12345
Δ. Λάθος χρόνου εκτέλεσης
Ε. Λάθος χρόνου μεταγλώττισης
```

Απάντηση/Λύση

Το j είναι τοπική μεταβλητή στο μπλοκ της do. Επομένως η εμβέλειά του περιορίζεται σε αυτό το μπλοκ. Η συνθήκη ελέγχου στην while βρίσκεται εκτός του μπλοκ της do και αναφέρεται στο j. Το j όμως σε εκείνη την περιοχή του κώδικα είναι αθέατο στον μεταγλωττιστή ο οποίος παράγει κατάλληλο μήνυμα λάθους. Επομένως, η σωστή απάντηση είναι η Ε.

Μεταβλητή ελέγχου της for

Έστω ο κώδικας

```
public class KA03 {

     public static void main(String[] args) {
          int i = 5;
          for (i = 10; i > 0; i--) {
               System.out.print(--i);
          }
          System.out.println(i);
     }
}
```

Ποια είναι η έξοδός του;

```
Α. 975310
Β. 987654321
Γ. 97531
Δ. Λάθος χρόνου εκτέλεσης
Ε. Λάθος χρόνου μεταγλώττισης
```

Απάντηση/Λύση

Η μεταβλητή ελέγχου της for ορίζεται μια φορά πριν τον βρόχο, οπότε μπορεί να χρησιμοποιηθεί χωρίς πρόβλημα. Έτσι, λάθος χρόνου εκτέλεσης δεν υπάρχει, άρα η απάντηση Δ δεν ισχύει. Επίσης, η απάντηση Ε δεν είναι σωστή, καθώς δεν υφίσταται κάποιο λάθος. Η απάντηση Β επίσης αποκλείεται, καθώς το i μειώνεται δύο φορές σε κάθε επαναληπτικό βήμα. Η απάντηση Γ είναι επίσης λανθασμένη. Η έξοδος 97531 πραγματοποιείται ως αποτέλεσμα του βρόχου. Ωστόσο, μετά το πέρας του βρόχου εκτελείται η System.out.println(i) η οποία τυπώνει το 0. Προσέξτε πως μετά την εκτύπωση του 1 από το σώμα της for, ο έλεγχος μεταβαίνει στην πρόταση της for όπου το i μειώνεται και γίνεται 0. Στη συνέχεια, η συνθήκη ελέγχου προκαλεί τερματισμό της for και ο κώδικας εκτελεί την τελική println με τιμή του i ίση με 0. Επομένως σωστή απάντηση είναι η Α.

Αρχικοποίηση τοπικών μεταβλητών

Έστω ο κώδικας

```
public class KA04 {
     public static void main(String[] args) {
          int x, y=5;
          if (y>5 || x>=Integer.MIN_VALUE) {
               x=y;
          }
          else {
```

```
                x=-y;
            }
            System.out.println(x*y);
        }
}
```

Ποια είναι η έξοδός του;

```
A. 25
B. -25
Γ. Λάθος χρόνου μεταγλώττισης
```

Απάντηση/Λύση

Στη συνθήκη ελέγχου της if επιχειρείται προσπέλαση της x που είναι αναρχικοποίητη τοπική μεταβλητή. Αυτό έχει ως αποτέλεσμα την παραγωγή λάθους χρόνου μεταγλώττισης. Επομένως σωστή είναι η απάντηση Γ.

Πράξεις με ακέραιους

Έστω ο κώδικας

```
public class KA05 {
     public static void main(String[] args) {
            short k=1;
            k=k+1;
            int m=k++ + 1;
            System.out.println(m+k);
     }
}
```

Ποια είναι η έξοδός του;

```
A. 5
B. 6
Γ. Λάθος χρόνου μεταγλώττισης
Δ. 7
Ε. Λάθος χρόνου εκτέλεσης
```

Απάντηση/Λύση

Στη δεύτερη γραμμή της main επιχειρείται η εκχώρηση k=k+1. Παρότι το k είναι short, η έκφραση k+1 είναι τύπου int. Καθώς ο int είναι μεγαλύτερου μεγέθους από τον short, η εκχώρηση αυτή δεν είναι επιτρεπτή, οπότε η σωστή απάντηση είναι η Γ.

Switch 1

Έστω ο κώδικας

```
public class KA06 {
     public static void main(String[] args) {
            int x=1;
            int c=++x+1;
            switch (c) {
                  case 0 : System.out.println(0);
                  case 1 : System.out.println(1);
                  case x : System.out.println(x);
```

```
                case 3 : System.out.println(c);
            }
        }
}
```

Ποια είναι η έξοδός του;

```
Α. 0123
Β. 123
Γ. 23
Δ. 3
Ε. Λάθος χρόνου μεταγλώττισης
```

Απάντηση/Λύση

Η τρίτη case της switch αναφέρεται στη μεταβλητή x. Στη θέση αυτή είναι αποδεκτές είτε κυριολεκτικές σταθερές είτε σταθερές μεταβλητές, δηλαδή μεταβλητές final. Η x δεν είναι ούτε το ένα, ούτε το άλλο. Επομένως, ο κώδικας παράγει λάθος χρόνου μεταγλώττισης και η σωστή απάντηση είναι η Ε.

Switch 2

Έστω ο κώδικας

```
public class KA07 {
     public static void main(String[] args) {
          int k=2;
          switch (k) {
               case 1: System.out.print(1);
               case 2: System.out.print(2);
               default: System.out.print(3);
          }
     }
}
```

Ποια είναι η έξοδός του;

```
Α. 1
Β. 2
Γ. 12
Δ. 3
Ε. 23
```

Απάντηση/Λύση

Η μεταβλητή k έχει την τιμή 2. Επομένως, η ροή του κώδικα κατευθύνεται στην case 2. Στη συνέχεια εκτελούνται οι προτάσεις μέχρι το τέλος της switch ή μέχρι το επόμενο break. Στο παράδειγμα αυτό δεν έχουμε break επομένως εμφανίζεται το 2 και στην συνέχεια το 3. Η σωστή απάντηση είναι η Ε.

Εκχώρηση και ισότητα

Έστω ο κώδικας

```
public class KA08 {
     public static void main(String[] args) {
          boolean b=7<=5;
          if (b=true) {
               System.out.println("7<=5");
```

```
            }
            else {
                  System.out.println("7>5");
            }
      }
}
```

Ποια είναι η έξοδός του;

```
A. 7<=5
B. 7>5
Γ. Λάθος χρόνου μεταγλώττισης
Δ. Λάθος χρόνου εκτέλεσης
```

Απάντηση/Λύση

Δεν υπάρχει λάθος χρόνου μεταγλώττισης, ούτε λάθος χρόνου εκτέλεσης. Προσέξτε πως στην if δεν χρησιμοποιείται ο τελεστής ισότητας αλλά ο τελεστής εκχώρησης. Έτσι το b δεν συγκρίνεται με το true αλλά εκχωρείται το true στο b. Η τιμή επιστροφής της εκχώρησης επομένως είναι true με αποτέλεσμα ο κώδικας να εισέρχεται στο μπλοκ της if και να εμφανίζει 7<=5. Η σωστή απάντηση επομένως είναι η Α.

Διαίρεση

Έστω ο κώδικας

```
public class KA09 {
      public static void main(String[] args) {
            System.out.println(3/2+""+3.0/2.0);
      }
}
```

Ποια είναι η έξοδός του;

```
A. 1.5 1.5
B. 1 1
Γ. 11.5
```

Απάντηση/Λύση

Το 2 και το 3 είναι κυριολεκτικές σταθερές ακέραιου τύπου. Επομένως και η διαίρεση μεταξύ τους είναι η ακέραια διαίρεση, δηλαδή 3/2=1. Αντίστοιχα, 3.0 και 2.0 είναι πραγματικές σταθερές και η μεταξύ τους διαίρεση είναι διαίρεση μεταξύ πραγματικών, δηλαδή 3.0/2.0=1.5. Δεδομένου ότι η προτεραιότητα της διαίρεσης είναι μεγαλύτερη από την προτεραιότητα της σύνδεσης-πρόσθεσης, η έξοδος του κώδικα είναι 11.5 και σωστή είναι η απάντηση Γ.

Διαίρεση και Σύνδεση

Έστω ο κώδικας

```
public class KA10 {
      public static void main(String[] args) {
            double i=1, j=0;
            System.out.print("sum equals to "+i+j);
            if (i/j> 1000) {
                  System.out.println("-Very Big");
            }
```

```
            else {
                  System.out.println("-Normal");
            }
      }
}
```

Ποια είναι η έξοδός του;

```
A. sum equals to 1-VeryBig
B. sum equals to 1-Normal
Γ. sum equals to 10-VeryBig
Δ. sum equals to 10-Normal
Ε. Λάθος χρόνου εκτέλεσης
ΣΤ. Τίποτε από τα παραπάνω
```

Απάντηση/Λύση

Η πρώτη print εμφανίζει sum equals to 10. Η έκφραση "sum equals to"+i+j αξιολογείται από αριστερά προς τα δεξιά. Η αλφαριθμητική σειρά "sum equals to " συνδέεται με το i το οποίο μετατρέπεται αυτόματα σε String. Το αποτέλεσμα "sum equals to 1" συνδέεται στη συνέχεια με το j. Στη συνέχεια, καθώς τα i και j είναι τύπου double, η διαίρεση i/j δεν προκαλεί λάθος χρόνου εκτέλεσης, αλλά επιστρέφει Infinite που θεωρείται μεγαλύτερο από το 1000, οπότε η συνθήκη της if αληθεύει και εμφανίζεται το -VeryBig. Επομένως, η σωστή απάντηση είναι η Γ.

Όρια Πίνακα 1

Έστω ο κώδικας

```
import java.util.Random;

public class KA11 {
      public static void main(String[] args) {
            Random g=new Random();
            int[] t={1,2,3,4,5,6};
            int idx=g.nextInt(6)+1;
            System.out.println(t[idx]);
      }
}
```

Ποια είναι η έξοδός του;

```
A. Τυπώνει έναν αριθμό από το 1 έως το 6
B. Τυπώνει έναν αριθμό από το 2 έως το 6
Γ. Λάθος χρόνου εκτέλεσης
Δ. Λάθος χρόνου μεταγλώττισης
```

Απάντηση/Λύση

Η συνάρτηση nextInt(6) επιστρέφει έναν ακέραιο από το 0 έως το 5. Επομένως, οι πιθανές τιμές για τη μεταβλητή idx είναι από 1 έως 6. Παράλληλα το μήκος του πίνακα t είναι 6. Κατά συνέπεια, οι δείκτες του πίνακα λαμβάνουν τιμές στην περιοχή από 0 έως 5. Επομένως, αν η τιμή της idx είναι από 1 έως 5, η συνάρτηση θα τυπώσει έναν αριθμό από το 2 έως το 6. Ωστόσο, ο κώδικας δεν εγγυάται αυτές τις τιμές για το idx. Αντίθετα, όπως εξηγήσαμε παραπάνω, η τιμή 6 είναι πιθανή για την idx. Σε αυτήν την περίπτωση, στην τελευταία πρόταση επιχειρούμε να προσπελάσουμε έξω από τα όρια του πίνακα. Αυτό θα προκαλέσει λάθος χρόνου εκτέλεσης. Η σωστή απάντηση είναι η Γ.

Όρια Πίνακα 2

Έστω ο κώδικας

```
public class KA12 {

     public static void main(String[] args) {
          int[][] t2Dim = {
           {1, 2, 3, 4},
           {5, 6, 7, 8},
           {9, 10, 11, 12}
          };
          int[] t={1,0,2,3};
          for (int i=0; i<t.length; i+=2) {
               System.out.print(t2Dim[t[i]][t[i]]);
          }
     }
}
```

Ποια είναι η έξοδός του;

```
A. 15
B. 17
Γ. 19
Δ. 111
E. 611
ΣΤ. Λάθος χρόνου μεταγλώττισης
```

Απάντηση/Λύση

Η for στην main ξεκινά από 0 και αυξάνεται με βήμα 2 έως ότου δεν υπερβεί το μήκος του πίνακα. Επομένως, η πρώτη τιμή της μεταβλητής i είναι το 0 και η δεύτερη το 2. Κατά συνέπεια, η t[i] για i=0 είναι ίση με 1 και η t[i] για i=2 είναι ίση με 2. Έτσι, ισχύει t2Dim[t[i]][t[i]]=t2Dim[1][1] για i=0 και t2Dim[t[i]][t[i]]=t2Dim[2][2]. Το μεν t2Dim[1][1] είναι ίσο με 6, το δε t2Dim[2][2] είναι ίσο με 11. Λάθος χρόνου μεταγλώττισης δεν υπάρχει, επομένως, η έξοδος είναι 611 και η σωστή απάντηση είναι η Ε.

Αναδρομή

Έστω ο κώδικας

```
public class KA13 {
     static void prt(int i) {
          System.out.print(i);
          if (i>=10)
               return;
          if (i%2==0) {
               prt(i+1);
          }
          else {
               prt(i+3);
          }
     }

     public static void main(String[] args) {
          prt(0);
     }
```

```
}
```

Ποια είναι η έξοδός του;

```
A. 01458912
B. 014589
Γ. 0347811
Δ. 03478
Ε. Καμία από τις παραπάνω
```

Απάντηση/Λύση

Κατά την κλήση prt(0) καταρχάς εμφανίζεται το 0. Στη συνέχεια, η συνθήκη ελέγχου i>=10 είναι ψευδής, ενώ η συνθήκη ελέγχου i%2==0 είναι αληθής, οπότε καλείται η prt(1). Η prt(1) αφού τυπώσει 1, καλεί την prt(4). Παρόμοια, η prt(4) εμφανίζει το 4 και καλεί prt(5). Εμφανίζεται το 5 και καλείται η prt(8). Εμφανίζεται το 8 και καλείται prt(9). Εμφανίζεται το 9 και καλείται η prt(12). Αυτή εμφανίζει πρώτα το 12 και στη συνέχεια βρίσκει false την έκφραση i>=10, οπότε τερματίζεται η αναδρομική κλήση. Καθώς από την επιστροφή από την αναδρομική κλήση, η prt δεν έχει άλλο κώδικα να εκτελέσει, η prt(0) τερματίζεται οριστικά. Επομένως, η έξοδος της prt(0) είναι 01458912 και σωστή απάντηση είναι η Α.

Αμοιβαία Αναδρομή

Έστω ο κώδικας

```
public class KA14 {

     static int f(int i) {
          if (i > 0) {
               return g(i);
          }
          return 0;
     }

     static int g(int i) {
          System.out.print(i);
          return f(i - 1);
     }

     public static void main(String[] args) {
          System.out.print(g(3));
     }
}
```

Ποιες από τις παρακάτω προτάσεις είναι σωστές;

```
A. Η έξοδος είναι 3210
B. Η g καλείται 3 φορές
Γ. Η f καλείται 3 φορές
Δ. Η έξοδος της κλήσης g(3) είναι 321
```

Απάντηση/Λύση

Η κλήση g(3) εμφανίζει 3 και καλεί την f(2). Η f(2) καλεί την g(2). Η g(2) εμφανίζει 2 και καλεί την f(1). Η f(1) καλεί την g(1) που εμφανίζει 1 και καλεί την f(0). Η f(0) επιστρέφει 0 και τερματίζει την αναδρομική κλήση. Επομένως, η g καλείται 3 φορές, το ίδιο και η f. Άρα οι προτάσεις Β και Γ είναι σωστές. Επιπλέον, η

έξοδος της g(3) είναι 321, άρα και η πρόταση Δ είναι σωστή. Τέλος, η τιμή επιστροφής της g(3) είναι 0. Η main εμφανίζει την τιμή αυτή, επομένως η έξοδος του κώδικα είναι 3210, δηλαδή και η πρόταση Α είναι σωστή.

Κλάση Math

Έστω ο κώδικας

```
public class KA15 {
     public static void main(String[] args) {
          float f;
          for (f=0; f<=9; f++); //γραμμή 4
          f=Math.sqrt(f) + 1;   //γραμμή 5
          f=f/1;
          System.out.println(f);
     }
}
```

Ποια από τις παρακάτω προτάσεις είναι σωστή;

```
Α. Η main έχει λάθος χρόνου μεταγλώττισης στη γραμμή 4
Β. Η main έχει λάθος χρόνου μεταγλώττισης στη γραμμή 5
Γ. Η main τυπώνει 4
Δ. Καμία από τις παραπάνω
```

Απάντηση/Λύση

Στη γραμμή 4 δεν υπάρχει λάθος. Όπως έχουμε εξηγήσει, κανένα από τα τμήματα της for δεν είναι υποχρεωτικό. Στη γραμμή 5, η sqrt επιστρέφει τιμή τύπου double, ενώ το f είναι float. Καθώς ο float είναι μικρότερης χωρητικότητας από τον double, η εκχώρηση αυτή δεν επιτρέπεται. Επομένως, η γραμμή 5 έχει λάθος χρόνου μεταγλώττισης. Κατά συνέπεια, ο κώδικας δεν μεταγλωττίζεται, άρα δεν τυπώνει 4. Η σωστή απάντηση είναι η Β.

Κλάση Boolean

Έστω ο κώδικας

```
public class KA16 {
     public static void main(String[] args) {
          int i=1, j=0;
          boolean b= i++>j && i>j+1; //2
          System.out.println(b                                    &&
          Boolean.parseBoolean("TRue"));
     }
}
```

Ποια είναι η έξοδός του;

```
Α. false
Β. true
Γ. Περιέχει λάθος χρόνου εκτέλεσης
```

Απάντηση/Λύση

Η έκφραση στη γραμμή 2 αξιολογείται ως εξής: Πρώτα αξιολογείται η έκφραση στα δεξιά της εκχώρησης, δηλαδή η έκφραση i++>j && i>j+1. Η αξιολόγηση εδώ γίνεται από αριστερά προς τα δεξιά. Η i++>j αυξάνει το i σε 2 και είναι αληθής, επομένως θα αξιολογηθεί και η i>j+1 η οποία επίσης είναι αληθής. Επομένως, η μεταβλητή b λαμβάνει την τιμή true. Η parseBoolean επιστρέφει επίσης true όταν το όρισμά της είναι η σειρά

true ανεξαρτήτως πεζών ή κεφαλαίων. Το λογικό and μεταξύ δύο εκφράσεων που είναι αληθείς επιστρέφει true. Λάθος χρόνου εκτέλεσης δεν υπάρχει. Συνεπώς, η main εμφανίζει true και η σωστή απάντηση είναι η Β.

Συστήματα Αρίθμησης

Έστω ο κώδικας

```
public class KA17 {
     public static void main(String[] args) {
          int i=010, j=011;
          System.out.println(i+j);
     }
}
```

Ποια είναι η έξοδός του;

```
Α. 21
Β. 17
Γ. Περιέχει λάθος χρόνου εκτέλεσης
Δ. Περιέχει λάθος χρόνου μεταγλώττισης
```

Απάντηση/Λύση

Όπως έχουμε αναφέρει στην ενότητα 3.6, οι κυριολεκτικές ακέραιες σταθερές που αρχίζουν από μηδέν εκφράζονται στο οκταδικό σύστημα. Ο 010 είναι η βάση του οκταδικού συστήματος, δηλαδή το 8. Εύκολα, μπορεί κανείς να καταλάβει πως ο 011 είναι ο επόμενος ακέραιος από τον 010, δηλαδή το 9. Επομένως, η έξοδος του κώδικα είναι 17 και η σωστή απάντηση η Β.